

MH1301 Discrete Mathematics – Revision Notes

Academic Year 2025–2026

Quantitative Research Society @ NTU

August 17, 2026

Abstract

These revision notes are organised around the official MH1301 handout sequence for AY2025/2026. The main body is split by part and handout, while the appendix collects compact reference material: notation, core identities, proof patterns, revision checklists, and common pitfalls. Chapter-level mathematical development remains inside dedicated chapter files so that the project can grow cleanly across the whole module.

Contents

How to Use This Document	5
Course Roadmap	10
Appendix	11
A Formula & Key Theorems Reference	11
A.1 Counting Fundamentals	11
A.1.1 Product Rule	11
A.1.2 Sum Rule	11
A.1.3 Inclusion–Exclusion (Two Sets)	11
A.1.4 Inclusion–Exclusion (Three Sets)	11
A.1.5 General Inclusion–Exclusion	11
A.1.6 Counting All Functions	11
A.1.7 Injective / Surjective / Bijective Constraints	11
A.1.8 Division Rule	11
A.1.9 Binary Strings	11
A.1.10 Subsets of an n -Element Set	12
A.2 Pigeonhole Principle	12
A.2.1 Basic Form	12
A.2.2 Generalized Form	12
A.3 Permutations, Combinations, and Binomial Identities	12
A.3.1 Permutations of n Distinct Objects	12
A.3.2 r -Permutations	12
A.3.3 r -Combinations	12
A.3.4 Choose–Then–Order Identity	12
A.3.5 Symmetry of Binomial Coefficients	12
A.3.6 Pascal’s Identity	12
A.3.7 Binomial Theorem	13
A.3.8 Coefficient in a Binomial Expansion	13
A.3.9 Sum of Binomial Coefficients	13
A.3.10 Cyclic Arrangements	13
A.3.11 Ordered Selections with Repetition	13
A.3.12 Unordered Selections with Repetition	13
A.3.13 Stars and Bars / Nonnegative Integer Solutions	13
A.3.14 Permutations of Indistinguishable Objects	14
A.3.15 Specified Occupancy Distribution	14
A.4 Recurrence Relations	14
A.4.1 General Form	14
A.4.2 Standard Modelling Examples	14
A.4.3 Linear Homogeneous Recurrence with Constant Coefficients	14
A.4.4 Characteristic Equation	14
A.4.5 Degree 2: Distinct Roots	14
A.4.6 Degree 2: Repeated Root	14
A.4.7 Degree k : Distinct Roots	15
A.4.8 Multiplicity Form	15
A.4.9 Non-Homogeneous Recurrence	15
A.4.10 Associated Homogeneous Recurrence	15

A.4.11	Structure Theorem	15
A.4.12	Polynomial Forcing Rule	15
A.4.13	Exponential Forcing Rule	15
A.5	Graph Theory	15
A.5.1	Simple Graph Edge Bound	15
A.5.2	Degree Sum Formula / Handshaking Lemma	16
A.5.3	Parity of Odd-Degree Vertices	16
A.5.4	Complete Graph	16
A.5.5	Cycle Graph	16
A.5.6	Subgraph and Induced Subgraph	16
A.5.7	Graph Isomorphism	16
A.5.8	Graph Invariants	16
B	Special Techniques (Beyond Syllabus but Useful)	16
B.1	Counting	16
B.1.1	Binary Encoding of Subsets	16
B.1.2	Count the Complement	17
B.1.3	Partition into Disjoint Cases Before Adding	17
B.1.4	Choose Then Order	17
B.1.5	Count First, Then Divide	17
B.1.6	Stars and Bars with Lower Bounds	17
B.1.7	Inclusion–Exclusion for Surjections	17
B.1.8	Coefficient Targeting	17
B.1.9	Worst-Case Pigeonhole Modelling	18
B.2	Recurrence Relations	18
B.2.1	Define the State Precisely	18
B.2.2	Split by the Last Step / Last Symbol	18
B.2.3	Use Auxiliary States When One State Is Awkward	18
B.2.4	Separate Homogeneous Part from Forcing Term	18
B.2.5	Diagnose Root Structure Before Writing the General Form	18
B.2.6	Resonance Test for Particular Solutions	18
B.2.7	Use Initial Conditions Last	18
B.3	Graph Theory	18
B.3.1	Ignore the Drawing; Read the Adjacency	18
B.3.2	Use the Handshaking Lemma Early	19
B.3.3	Use Parity Arguments	19
B.3.4	Use Extremal Degree Arguments	19
B.3.5	Disprove Isomorphism with Invariants First	19
B.3.6	Build Isomorphisms by Matching Rigid Structure	19
C	Notation Summary	20
D	Key Counting Identities	20
E	Recurrence Solving Checklist	21
F	Graph Theory Theorem Checklist	21
G	Proof Strategies Across the Module	21
H	Common Pitfalls	22

I One-Page Revision Workflow**22**

QRS@NTU — qrsntu.org

How to Use This Document

1. Purpose of the notes

These notes are designed to be more than a typed copy of lecture material and less than a full textbook. Their purpose is to give a *revision-oriented working version* of MH1301: each handout is reorganised so that definitions, standard counting mechanisms, proof patterns, and common traps appear in a form that is useful both during the semester and during exam preparation.

The guiding principle is that discrete mathematics is not well revised by memorising isolated answers. Most questions in this module are solved by choosing a correct representation of the objects being counted or analysed. For that reason, the notes repeatedly emphasise the following workflow:

identify the objects $\longrightarrow$ decide the right model
 $\longrightarrow$ choose the right counting/proof tool
 $\longrightarrow$ check edge cases and overlap.

A useful way to think about the document is the following.

- The **main exposition** tells you what the formal statements mean and when they should be used.
- The **worked examples** show how those statements appear in actual MH1301-style questions.
- The **common mistake** boxes tell you where students usually lose marks.
- The **exam focus** boxes isolate high-yield checks, recognition patterns, or revision routines.
- The **extension** boxes provide concise structural viewpoints that are valuable once the core syllabus statement is already understood.

Used properly, the notes should help you move from passive recognition (“I have seen this formula before”) to active control (“I can recognise the type of problem, justify the method, and reproduce the argument myself”).

2. Structure by part and handout

The document follows the module handout structure rather than reorganising the entire course by topic. This is deliberate. In revision, it is useful to preserve the same broad sequencing as lectures while making the internal structure of each handout more explicit.

- **Part I: Counting.** These handouts develop the modelling language of the course: product rule, sum rule, inclusion–exclusion, permutations, combinations, binomial ideas, and related methods. Much of the rest of the module still relies on the same discipline of representing objects carefully.
- **Part II: Recurrence Relations.** These handouts shift from static counting formulas to recursively defined sequences. When revising this part, it is especially important to distinguish the recurrence itself, the initial conditions, and the method used to derive a closed form.
- **Part III: Graph Theory.** These handouts introduce graph language, degree arguments, connectivity, paths, cycles, and trees. Here the emphasis moves from counting choices to verifying definitions and applying structural theorems correctly.

Within each chapter file, the intended reading order is usually:

1. definitions and basic notation;
2. main theorems or counting rules;
3. short examples showing immediate use;
4. higher-value techniques or interpretations;
5. end-of-section reminders and pitfalls.

This order matters. If you read examples without first knowing exactly what a theorem says, you tend to remember the surface story of the example rather than the reusable method.

3. How to read the different box types

These notes use a box system to separate different kinds of information. The boxes are not decorative; they are intended to change how you read.

Box type	How to use it
Definition	Read slowly enough that you could restate the definition in your own words and also recognise a disguised use of it in a question.
Theorem	Learn both the formal statement and the trigger for using it. A theorem is useful only when you can recognise its hypotheses in a new problem.
Key Formula	Use as a compact memory anchor only after you understand why the formula is true and what assumptions make it valid.
Worked Example	Cover the solution first and try to predict the method. When reading the solution, focus on the decision points rather than the final algebra.
Common Mistake	Treat as a pre-commitment check. Before attempting tutorial questions, scan the nearby mistake boxes and actively avoid those errors.
Exam Focus	Use as a high-yield checklist. These boxes are especially useful in the week before a test, when you want rapid reminders of recognition patterns and final checks.
Extension	Read only after the core argument is secure. Extensions are intended to sharpen understanding, not replace the main route through the material.

A common bad habit is to treat boxed material as content to memorise and ordinary paragraphs as content to skip. That is usually backwards. The ordinary paragraphs explain why the box exists, what kind of question it answers, and what limitation it has.

4. Four stages of revision

First learning. During the first pass through a handout, the goal is not speed. Read definitions carefully, rewrite the main theorem statements by hand, and pause before each worked example to ask what information in the question tells you which tool is appropriate. At this stage, the notes should be read with the lecture slides or tutorial sheet nearby.

Weekly consolidation. At the end of each teaching week, revisit the relevant section and reduce it to a shorter personal summary. For counting material, list the standard recognition cues: when a problem is sequential, when cases are disjoint, when overlap forces inclusion–exclusion, when a bijection is more natural than direct counting, and when complement counting is cleaner than direct counting. For recurrence material, summarise which class of recurrence you have seen and which solving mechanism belongs to it. For graph theory, list the definitions and theorem triggers that are easiest to confuse.

Pre-midterm review. At this stage the notes should no longer be read linearly from page 1 onward. Instead, revise by question type. For example, in the counting part, group together all questions that rely primarily on product rule, all questions that rely on sum rule with disjoint partition, and all questions where overlap detection is the real issue. Use the worked examples as checkpoints, not as reading material. You should be able to reconstruct their structure without copying them line by line.

Pre-final review. Before the final, the priority is integration. You should be able to move across parts of the course and identify differences in proof style. Counting questions often demand careful modelling; recurrence questions demand algebra plus indexing discipline; graph theory questions demand precise use of definitions and theorem hypotheses. At this stage, the notes are most useful when used diagnostically: identify which theorem statements you can still recognise but not reproduce, which examples you understand only after reading, and which common mistakes you still make under time pressure.

A productive revision cycle

For any section, a strong short revision cycle is:

1. read the definitions and theorem statements once without looking at examples;
2. close the notes and restate the main ideas from memory;
3. attempt one tutorial-style question cold;
4. reopen the notes only to check the exact step where your reasoning diverged.

This is substantially more effective than rereading the same worked solution several times.

5. How to read theorem statements productively

A theorem should not be read as a sentence to be memorised mechanically. For revision purposes, every theorem in MH1301 should be read with four questions in mind:

1. **What are the hypotheses?** What must be true before the theorem can be used?
2. **What is the conclusion?** What exactly does the theorem allow me to claim?
3. **What class of problem does this theorem solve?** In practice, what does a question look like when this theorem is the right tool?
4. **What goes wrong if a hypothesis fails?** Is there a standard overcounting, missing-case, or false-conclusion trap?

For example, the sum rule is not “add counts whenever you see an *or*.” It is a statement about *disjoint* unions. Similarly, inclusion–exclusion is not merely a correction formula to memorise; it is the right response when classes overlap in a controlled way. Reading the theorem with its hypotheses foregrounded prevents many routine mistakes.

6. How to use worked examples without becoming passive

Worked examples are most valuable when they are used actively. A good procedure is:

1. Cover the solution.
2. Read only the question and identify the combinatorial objects involved.
3. Decide which rule, theorem, or representation is most likely to be relevant.
4. Write the first two or three lines yourself.
5. Only then compare with the notes.

When comparing with the written solution, do not ask only whether the final answer matches. Ask instead:

- Did I partition the cases in the same way?
- Did I justify why the chosen rule applies?
- Did I ignore an overlap or edge case?
- Could I produce the same argument again tomorrow without seeing it?

If an example feels easy only when the solution is visible, then it is not yet learned.

7. How to use special techniques and extensions properly

Special techniques and extension boxes are included because exam-level understanding often requires more than the shortest syllabus statement. However, they should be used in the right order. The core path is always:

definition $\longrightarrow$ main theorem/rule $\longrightarrow$ standard example.

Only after that should you use the extensions to deepen or compress your thinking.

Typical uses of extension material include:

- seeing why a bijection is canonical rather than ad hoc;
- understanding inclusion–exclusion through indicator-style reasoning;
- recognising a more elegant derivation after mastering the standard derivation;
- seeing how one theorem packages several simpler observations into one structural statement.

Used properly, extensions improve retention because they connect ideas. Used too early, they can make the notes look more advanced than they are. The right test is simple: if you cannot yet do the core example unassisted, return to the main exposition first.

8. Do not treat the notes as a formula sheet

These notes contain formulas and compact summaries, but MH1301 is not a formula-substitution module. Many wrong answers in discrete mathematics arise not because a formula is forgotten, but because the student counts the wrong set, assumes cases are disjoint when they are not, forgets that order matters, or applies a theorem outside its valid setting.

For that reason, every time you use a formula, you should still ask:

- What set is being counted?
- Why does this object decompose into stages or cases in the way I claim?
- Is the representation ordered or unordered?
- Is repetition allowed?
- Are the cases disjoint, or must I correct overlap?

The formula then becomes the final compression of a correct model, not a substitute for one.

9. How to cross-check understanding

A reliable test of understanding is whether you can reproduce an argument without immediately looking back at the notes. Useful self-checks include:

- restating a theorem from memory and identifying its hypotheses;
- reconstructing the proof idea of a short result, even if not every word matches the written version;
- solving a nearby tutorial question using the same core method but with different numbers or conditions;
- explaining aloud why a tempting incorrect method fails.

In practical use, the notes should be paired with tutorial questions, short self-quizzes, and timed practice. After reading a section, attempt at least one question for which you do *not* already know the answer. The objective is not to finish the notes; it is to become able to produce the reasoning independently.

Using these notes at high intensity near an exam

When time is short, do not reread whole chapters uniformly. Instead, scan the theorem statements and exam-focus boxes, then choose only the sections where you cannot immediately answer the question: *Could I reconstruct the main argument here on a blank page?* This triages revision toward genuine weaknesses rather than familiar pages.

Course Roadmap

Part I: Counting

- Handout 01 — Basics of Counting
- Handout 02 — Pigeonhole Principle, Permutations and Combination
- Handout 03 — Binomial Theorem, Generalized Permutations and Combination

Part II: Recurrence Relations

- Handout 04 — Recurrence Relations
- Handout 05 — Linear Homogeneous Recurrence Relations
- Handout 06 — Linear Non-Homogeneous Recurrence Relations

Part III: Graph Theory

- Handout 07 — Graph Theory (1)
- Handout 08 — Graph Theory (2)
- Handout 09 — Graph Theory (3)
- Handout 10 — Graph Theory (4)
- Handout 11 — Graph Theory (5)
- Handout 12 — Trees

What changes across the three parts

In Part I, most questions are solved by modelling and counting. In Part II, the main challenge is setting up and solving recurrences with the correct indexing and initial conditions. In Part III, precision of definitions becomes even more important: many graph questions are easy to state informally but require exact theorem use to solve cleanly.

A Formula & Key Theorems Reference

A.1 Counting Fundamentals

A.1.1 Product Rule

If an object is built in k stages with $n_1, n_2, \dots, n_k$ valid choices at each stage, then

$$N = n_1 n_2 \cdots n_k.$$

A.1.2 Sum Rule

If $A_1, \dots, A_k$ are pairwise disjoint finite sets, then

$$|A_1 \cup \cdots \cup A_k| = |A_1| + \cdots + |A_k|.$$

A.1.3 Inclusion–Exclusion (Two Sets)

$$|A \cup B| = |A| + |B| - |A \cap B|.$$

A.1.4 Inclusion–Exclusion (Three Sets)

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|.$$

A.1.5 General Inclusion–Exclusion

For finite sets $A_1, \dots, A_n$,

$$\left| \bigcup_{i=1}^n A_i \right| = \sum_i |A_i| - \sum_{i < j} |A_i \cap A_j| + \sum_{i < j < k} |A_i \cap A_j \cap A_k| - \cdots + (-1)^{n+1} \left| \bigcap_{i=1}^n A_i \right|.$$

A.1.6 Counting All Functions

If $|X| = m$ and $|Y| = n$, then the number of functions $f : X \rightarrow Y$ is

$$n^m.$$

A.1.7 Injective / Surjective / Bijective Constraints

If $f : X \rightarrow Y$ with $|X| = m$, $|Y| = n$, then

$$f \text{ injective} \implies m \leq n, \quad f \text{ surjective} \implies m \geq n, \quad f \text{ bijective} \implies m = n.$$

Also,

$$\#\{\text{injective } f : X \rightarrow Y\} = P(n, m) \quad (m \leq n), \quad \#\{\text{bijections } X \rightarrow Y\} = n! \quad (m = n).$$

A.1.8 Division Rule

If $f : X \rightarrow Y$ is k -to-1 and X, Y are finite, then

$$|Y| = \frac{|X|}{k}.$$

A.1.9 Binary Strings

The number of binary strings of length n is

$$2^n.$$

A.1.10 Subsets of an n -Element Set

$$\#\{A : A \subseteq S\} = 2^n \quad (|S| = n).$$

A.2 Pigeonhole Principle**A.2.1 Basic Form**

If $k + 1$ or more objects are placed into k boxes, then some box contains at least two objects.

A.2.2 Generalized Form

If N objects are placed into k boxes, then some box contains at least

$$\left\lceil \frac{N}{k} \right\rceil$$

objects.

A.3 Permutations, Combinations, and Binomial Identities**A.3.1 Permutations of n Distinct Objects**

$$n!.$$

A.3.2 r -Permutations

For $0 \leq r \leq n$,

$$P(n, r) = n(n-1) \cdots (n-r+1) = \frac{n!}{(n-r)!}.$$

A.3.3 r -Combinations

For $0 \leq r \leq n$,

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}.$$

A.3.4 Choose–Then–Order Identity

$$P(n, r) = \binom{n}{r} r!.$$

A.3.5 Symmetry of Binomial Coefficients

$$\binom{n}{r} = \binom{n}{n-r}.$$

A.3.6 Pascal's Identity

For $n \geq r \geq 1$,

$$\binom{n+1}{r} = \binom{n}{r} + \binom{n}{r-1}.$$

A.3.7 Binomial Theorem

For $n \in \mathbb{Z}_{\geq 0}$,

$$(x + y)^n = \sum_{i=0}^n \binom{n}{i} x^{n-i} y^i.$$

A.3.8 Coefficient in a Binomial Expansion

In

$$(x + y)^n = \sum_{i=0}^n \binom{n}{i} x^{n-i} y^i,$$

the coefficient of $x^{n-i} y^i$ is

$$\binom{n}{i}.$$

More generally, in $(ax + by)^n$, the coefficient of $x^{n-i} y^i$ is

$$\binom{n}{i} a^{n-i} b^i.$$

A.3.9 Sum of Binomial Coefficients

$$\sum_{i=0}^n \binom{n}{i} = 2^n.$$

A.3.10 Cyclic Arrangements

The number of ways to choose r elements from an n -element set and arrange them around a circle is

$$\frac{1}{r} P(n, r) = \frac{n!}{r(n-r)!}.$$

A.3.11 Ordered Selections with Repetition

The number of length- r strings from n symbols is

$$n^r.$$

A.3.12 Unordered Selections with Repetition

The number of r -combinations from n types with repetition allowed is

$$\binom{n+r-1}{r} = \binom{n+r-1}{n-1}.$$

A.3.13 Stars and Bars / Nonnegative Integer Solutions

The number of solutions in nonnegative integers to

$$x_1 + x_2 + \cdots + x_n = r$$

is

$$\binom{n+r-1}{r}.$$

A.3.14 Permutations of Indistinguishable Objects

If $n = n_1 + \cdots + n_k$, then the number of distinct permutations of a multiset with multiplicities $n_1, \dots, n_k$ is

$$\frac{n!}{n_1!n_2!\cdots n_k!}.$$

A.3.15 Specified Occupancy Distribution

The number of ways to distribute n distinct objects into k distinct boxes so that box i receives n_i objects ($n_1 + \cdots + n_k = n$) is

$$\frac{n!}{n_1!n_2!\cdots n_k!}.$$

A.4 Recurrence Relations

A.4.1 General Form

A recurrence relation of degree k has the form

$$a_n = F(a_{n-1}, a_{n-2}, \dots, a_{n-k}),$$

together with enough initial conditions to determine the sequence.

A.4.2 Standard Modelling Examples

$$H_1 = 1, \quad H_n = 2H_{n-1} + 1 \quad (\text{Tower of Hanoi}),$$

$$H_n = 2^n - 1,$$

$$f_0 = 0, \quad f_1 = 1, \quad f_n = f_{n-1} + f_{n-2} \quad (\text{Fibonacci / graduate-student model}),$$

$$a_1 = 2, \quad a_2 = 3, \quad a_n = a_{n-1} + a_{n-2} \quad (\text{bit strings with no consecutive zeros}),$$

$$a_0 = 1, \quad a_n = 8a_{n-1} + 10^{n-1} \quad (\text{decimal strings with an even number of zeros}).$$

A.4.3 Linear Homogeneous Recurrence with Constant Coefficients

A degree- k linear homogeneous recurrence has the form

$$a_n = c_1a_{n-1} + c_2a_{n-2} + \cdots + c_ka_{n-k}, \quad c_k \neq 0.$$

A.4.4 Characteristic Equation

Associate

$$r^k - c_1r^{k-1} - c_2r^{k-2} - \cdots - c_{k-1}r - c_k = 0.$$

A.4.5 Degree 2: Distinct Roots

If

$$r^2 - c_1r - c_2 = 0$$

has distinct roots r_1, r_2 , then

$$a_n = h_1r_1^n + h_2r_2^n.$$

A.4.6 Degree 2: Repeated Root

If the characteristic equation has repeated root r_0 , then

$$a_n = (h_1 + h_2n)r_0^n.$$

A.4.7 Degree k : Distinct Roots

If the characteristic equation has k distinct roots $r_1, \dots, r_k$, then

$$a_n = h_1 r_1^n + \dots + h_k r_k^n.$$

A.4.8 Multiplicity Form

If r_i is a root of multiplicity m_i , then the corresponding contribution is

$$(h_{i,0} + h_{i,1}n + \dots + h_{i,m_i-1}n^{m_i-1}) r_i^n.$$

A.4.9 Non-Homogeneous Recurrence

A degree- k linear non-homogeneous recurrence has the form

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \dots + c_k a_{n-k} + F(n).$$

A.4.10 Associated Homogeneous Recurrence

Remove the forcing term:

$$b_n = c_1 b_{n-1} + c_2 b_{n-2} + \dots + c_k b_{n-k}.$$

A.4.11 Structure Theorem

If $a_n^{(p)}$ is one particular solution, then every solution is

$$a_n = a_n^{(p)} + b_n,$$

where b_n is a solution of the associated homogeneous recurrence.

A.4.12 Polynomial Forcing Rule

If $F(n)$ is a polynomial of degree t , then try

$$a_n^{(p)} = p_t n^t + p_{t-1} n^{t-1} + \dots + p_0$$

if 1 is not a root of the characteristic equation, and try

$$a_n^{(p)} = n(p_t n^t + p_{t-1} n^{t-1} + \dots + p_0)$$

if 1 is a root.

A.4.13 Exponential Forcing Rule

If $F(n) = s^n$, then try

$$a_n^{(p)} = c s^n$$

if s is not a root of the characteristic equation, and try

$$a_n^{(p)} = c n s^n$$

if s is a root.

A.5 Graph Theory**A.5.1 Simple Graph Edge Bound**

If $G = (V, E)$ is a simple graph with $|V| = n$, then

$$0 \leq |E| \leq \binom{n}{2}.$$

A.5.2 Degree Sum Formula / Handshaking Lemma

For any finite undirected graph,

$$\sum_{v \in V} \deg(v) = 2|E|.$$

A.5.3 Parity of Odd-Degree Vertices

In any finite undirected graph, the number of vertices of odd degree is even.

A.5.4 Complete Graph

For K_n ,

$$\deg(v) = n - 1 \quad (\forall v), \quad |E(K_n)| = \binom{n}{2}.$$

A.5.5 Cycle Graph

For C_n ($n \geq 3$),

$$\deg(v) = 2 \quad (\forall v), \quad |E(C_n)| = n.$$

A.5.6 Subgraph and Induced Subgraph

If $H = (W, F)$ is a subgraph of $G = (V, E)$, then

$$W \subseteq V, \quad F \subseteq E.$$

If H is induced on W , then F contains all edges of G whose endpoints both lie in W .

A.5.7 Graph Isomorphism

Graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are isomorphic iff there exists a bijection

$$f : V_1 \rightarrow V_2$$

such that

$$\{u, v\} \in E_1 \iff \{f(u), f(v)\} \in E_2.$$

A.5.8 Graph Invariants

If $G_1 \cong G_2$, then invariants such as the following agree:

$$|V_1| = |V_2|, \quad |E_1| = |E_2|,$$

and the degree sequence is preserved.

B Special Techniques (Beyond Syllabus but Useful)**B.1 Counting****B.1.1 Binary Encoding of Subsets**

Encode $A \subseteq \{1, \dots, n\}$ by its characteristic string

$$(\chi_A(1), \dots, \chi_A(n)) \in \{0, 1\}^n.$$

This converts subset problems into bit-string problems.

B.1.2 Count the Complement

When “bad” objects are easier to count,

$$\#(\text{good}) = \#(\text{all}) - \#(\text{bad}).$$

Example:

$$\#(\text{at least one property}) = \#(\text{all}) - \#(\text{none}).$$

B.1.3 Partition into Disjoint Cases Before Adding

Use the sum rule only after producing a genuine partition. A common split is by length, by first position, by last symbol, or by whether a distinguished element is chosen.

B.1.4 Choose Then Order

When the final object is ordered but the underlying selected set is easier to count,

$$P(n, r) = \binom{n}{r} r!.$$

This is often the cleanest way to avoid overcounting.

B.1.5 Count First, Then Divide

If several linear descriptions represent the same final object, compute the easier linear count and divide by the symmetry factor. Examples:

$$\text{cyclic arrangements} = \frac{P(n, r)}{r}, \quad \text{multiset permutations} = \frac{n!}{n_1! \cdots n_k!}.$$

B.1.6 Stars and Bars with Lower Bounds

For

$$x_1 + \cdots + x_n = r, \quad x_i \geq \ell_i,$$

set

$$x_i = \ell_i + y_i, \quad y_i \geq 0.$$

Then

$$y_1 + \cdots + y_n = r - (\ell_1 + \cdots + \ell_n),$$

after which stars and bars applies.

B.1.7 Inclusion–Exclusion for Surjections

To count surjections $f : X \rightarrow Y$ with $|X| = m$, $|Y| = n$, exclude functions missing at least one codomain value:

$$\#\{\text{surjections } X \rightarrow Y\} = \sum_{j=0}^n (-1)^j \binom{n}{j} (n-j)^m.$$

This is a direct high-value application of inclusion–exclusion.

B.1.8 Coefficient Targeting

In binomial expansions, do not expand fully; match exponents directly. Example:

$$(2x - 3y)^{25} = \sum_{i=0}^{25} \binom{25}{i} (2x)^{25-i} (-3y)^i.$$

For $x^{12}y^{13}$, go straight to $i = 13$.

B.1.9 Worst-Case Pigeonhole Modelling

For existence problems, choose boxes so that avoiding the target phenomenon makes each box as full as possible. If the total still exceeds capacity, the desired conclusion is forced.

B.2 Recurrence Relations

B.2.1 Define the State Precisely

A recurrence is only as good as its state variable. State exactly what a_n counts or measures before writing any recurrence.

B.2.2 Split by the Last Step / Last Symbol

For combinatorial recurrences, classify objects by how they end. Example: for bit strings with no consecutive zeros,

$$a_n = a_{n-1} + a_{n-2},$$

according as the string ends in 1 or in 01.

B.2.3 Use Auxiliary States When One State Is Awkward

If a single recurrence is hard to write, split into complementary states. Example: even-zero / odd-zero codewords, then combine the two states afterward.

B.2.4 Separate Homogeneous Part from Forcing Term

For

$$a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k} + F(n),$$

first solve the associated homogeneous recurrence, then find one particular solution, then add them.

B.2.5 Diagnose Root Structure Before Writing the General Form

Do not write the solution template until the characteristic roots and multiplicities are known: distinct roots $\Rightarrow$ exponential sum, repeated roots $\Rightarrow$ multiply by powers of n .

B.2.6 Resonance Test for Particular Solutions

If the guessed particular form overlaps with the homogeneous solution, multiply by n . Typical cases:

$$1 \text{ root} \Rightarrow \text{polynomial forcing needs an extra } n,$$

$$s \text{ root} \Rightarrow s^n \text{ forcing needs an extra } n.$$

B.2.7 Use Initial Conditions Last

Keep the constants $h_1, h_2, \dots$ until the full general solution is assembled; only then substitute the initial values.

B.3 Graph Theory

B.3.1 Ignore the Drawing; Read the Adjacency

Two pictures may look different but define the same graph. Always reduce to vertex set, edge set, adjacency, and degree data.

B.3.2 Use the Handshaking Lemma Early

Whenever degrees and edges appear together, immediately test

$$\sum_{v \in V} \deg(v) = 2|E|.$$

This often gives the shortest contradiction or the missing parameter.

B.3.3 Use Parity Arguments

Since the total degree sum is even, odd-degree vertices must come in pairs. This is a standard quick contradiction device.

B.3.4 Use Extremal Degree Arguments

Examples: a simple graph on n vertices cannot contain both a degree-0 vertex and a degree- $(n - 1)$ vertex; hence n vertices realize at most $n - 1$ distinct degree values.

B.3.5 Disprove Isomorphism with Invariants First

Before attempting an explicit bijection, compare easy invariants:

$|V|$, $|E|$, degree sequence, number of isolated vertices, local adjacency patterns.

If any invariant differs, the graphs are not isomorphic.

B.3.6 Build Isomorphisms by Matching Rigid Structure

When graphs might be isomorphic, start with vertices of unique degree or unusual neighbourhood type, map those first, then extend the bijection to adjacent vertices.

C Notation Summary

Symbol	Meaning	Typical MH1301 use
$A, B, C, \dots$	sets	classes of objects, event-like subsets, graph vertex/edge sets
$ A $	cardinality of A	number of elements in a finite set
$A \cup B$	union	objects satisfying at least one of two conditions
$A \cap B$	intersection	objects satisfying both conditions
$A \setminus B$	set difference	complement-style counting inside a universe
A^c	complement of A	count the easier excluded set first
$\emptyset$	empty set	impossible class / no common element
$[n]$	$\{1, 2, \dots, n\}$	standard labelled finite set
$\mathcal{P}(S)$	power set of S	family of all subsets of S
$f : X \rightarrow Y$	function from X to Y	assignments, injections, surjections, bijections
$f^{-1}(y)$	fibre / preimage of y	surjectivity and image counting viewpoints
$n!$	factorial	permutations and arrangements
$\binom{n}{k}$	binomial coefficient	choosing k objects from n
$\sum$	finite sum	disjoint case counts or indexed totals
$\prod$	finite product	sequential independent choices
$V(G), E(G)$	vertex and edge sets of a graph	graph theory handouts
a_n	sequence term	recurrence relations

Notation errors are often conceptual errors

When a counting solution is unclear, the underlying problem is often that the objects were not defined precisely. Writing down the set, the domain/codomain of a function, or the indexing of a sequence is not cosmetic; it is part of the mathematical solution.

D Key Counting Identities

Core identities from Part I

For finite sets and standard counting situations,

$$|A \cup B| = |A| + |B| - |A \cap B|,$$

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|,$$

$$\#\{\text{functions } X \rightarrow Y\} = |Y|^{|X|}, \quad \#\{n\text{-bit strings}\} = 2^n, \quad |\mathcal{P}(S)| = 2^{|S|}.$$

Alternating signs in inclusion–exclusion

If an element belongs to exactly r of the sets $A_1, \dots, A_n$, then its total contribution to the inclusion–exclusion sum is

$$\binom{r}{1} - \binom{r}{2} + \binom{r}{3} - \dots + (-1)^{r-1} \binom{r}{r} = 1.$$

This is the structural reason the formula counts every element of the union exactly once.

E Recurrence Solving Checklist

1. Define the sequence and state its initial conditions explicitly.
2. Decide whether the recurrence is first-order, linear, homogeneous, or non-homogeneous.
3. For linear recurrences, write the characteristic equation carefully.
4. Distinguish distinct roots, repeated roots, and forcing terms.
5. Solve for the constants using the initial values.
6. Check the resulting formula against the first few terms.
7. If the recurrence came from counting, interpret the formula combinatorially to catch indexing mistakes.

Closed form without correct indexing

A formally correct characteristic-root calculation can still lead to a wrong answer if the indexing convention is misread. Always check whether the recurrence is stated from $n \geq 1$, $n \geq 2$, or another starting point, and match the initial data accordingly.

F Graph Theory Theorem Checklist

- Distinguish carefully between walk, trail, path, circuit, and cycle.
- Check the Handshaking Lemma early whenever degree information appears.
- Separate local degree constraints from global connectedness claims.
- In bipartite questions, test parity and odd-cycle obstructions.
- In tree questions, remember the standard equivalences: connected and acyclic; unique simple path; $n - 1$ edges.
- When proving impossibility, cite the necessary theorem rather than relying only on a picture.

Why graph theory revision feels different from counting revision

In counting, the main difficulty is often choosing the right representation. In graph theory, the main difficulty is often choosing the right *definition* or theorem trigger. Many graph questions become short once the correct structural fact is recognised.

G Proof Strategies Across the Module

Strategy	Typical use
Direct counting	build an object stage by stage and multiply
Disjoint partition	split a set into mutually exclusive classes and add
Complement counting	count the whole universe and subtract the easier bad set
Counting by bijection	replace awkward objects by easier coded objects
Two-way counting	count the same set in two different ways to obtain an identity
Induction	prove recurrence or general inclusion–exclusion style statements
Contradiction / obstruction	show that a configuration cannot exist, often in graph theory

What a polished MH1301 solution usually contains

A strong solution usually identifies the set or structure precisely, chooses a method for a reason, shows the main counting or proof step cleanly, and states the final result in a way that makes the role of hypotheses visible.

H Common Pitfalls

Order versus no order

A sequence and a set are not the same object. Passwords, strings, line-ups, and permutations are ordered objects. Subsets and committees are usually unordered unless the question says otherwise.

Applying the sum rule without disjointness

The sum rule is a statement about disjoint unions. If an object can satisfy two case descriptions simultaneously, raw addition overcounts and inclusion–exclusion or a finer partition is needed.

Counting functions as though they were permutations

If $|X| = m$ and $|Y| = n$, then the number of functions $X \rightarrow Y$ is n^m , because each domain element independently chooses one image. This is a different problem from arranging distinct elements without repetition.

Using complements without fixing the universe

The expression A^c is meaningless until the ambient universe is understood. When using complement counting, state clearly what the total universe is.

I One-Page Revision Workflow

A compact working checklist

Before finishing a solution, ask:

1. What exactly is the set, sequence, function, or graph object under discussion?
2. Does order matter?
3. Is repetition allowed?
4. Are the cases disjoint?
5. If not, where does overlap enter?
6. Is there a better coding or bijection?
7. Have I checked initial conditions, special cases, or impossible configurations?